

From Pixels to Patterns

From image numbers to a decision: explore each operation, test the idea, then trace a complete CNN.

Play, pause and step through the figures. Each section ends with two quick checks.

01 / THE INPUT

How can an image be numbers?

What does a computer actually receive when it sees a photo?

An image is a grid of pixels. A grayscale pixel has one intensity; an RGB pixel has red, green and blue values.

RGB values describe the same location. Change them to change that pixel’s colour.

PIXEL · ROW 4, COLUMN 4



Gray  50

Intensity = 50

8-bit values: 0–255.
Training often rescales them to 0–1.

220

50

50

220

50

50

220

220

220

Intensity

A 3 × 3 crop, split into channels

8 × 8 × 1 = 64 values

Shape = height × width × channels.
A channel is one grid of values.

Select a pixel to inspect it.

Image representation provides a numerical grid: height × width × channels.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · One RGB pixel contains...

☐ One value

☐ Three values

☐ Three locations

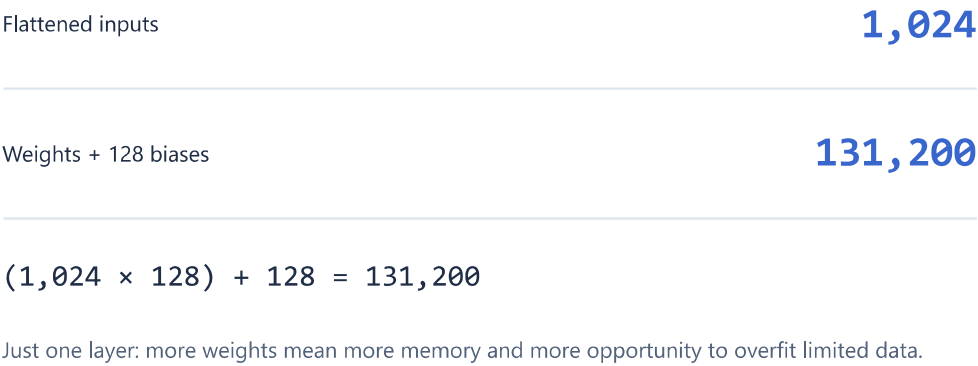
► Show reasoning

Numerical · 4 × 4 RGB: total values?

Answer

Flattening preserves the values, but a dense layer does not build in spatial neighborhoods or reuse one detector at every location.

2.1 Every pixel gets a separate weight



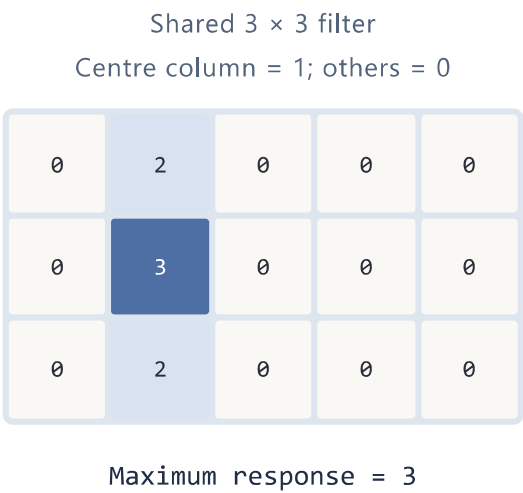
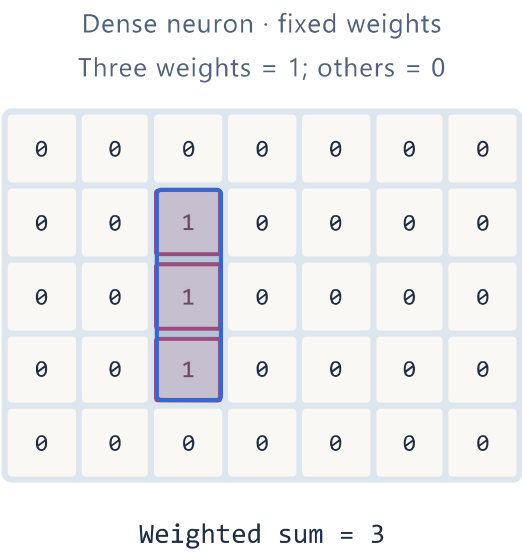
Every input connects to every dense neuron.

A dense layer does not privilege neighbouring pixels over distant ones. Convolution begins with a small patch, where local patterns form.

Slide the same detector everywhere. Its weights stay the same.

2.2 The same edge, in a different place

Dense weights belong to **fixed pixel positions**. Moving an edge changes which weights it meets. Convolution reuses one local detector everywhere.



Both respond with 3. Blue: input edge. Rose: fixed dense weights.

This illustrates one neuron, not a trained classifier. A convolution's response **moves with the pattern** (equivariance); the final prediction is not automatically invariant.

Three limitations: dense layers use many parameters, do not privilege nearby pixels, and do not automatically reuse a detector across locations. CNNs build in locality and sharing.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · MLP weights attach to...

- ☐ Pixel positions
- ☐ Any matching patch
- ☐ Image labels

Numerical · 4 inputs → 2 neurons: parameters?

Answer

► [Show reasoning](#)

First, a filter designed by hand.

How can nine numbers detect an edge?

Sobel filters detect intensity changes. One responds to vertical edges; the other responds to horizontal edges.

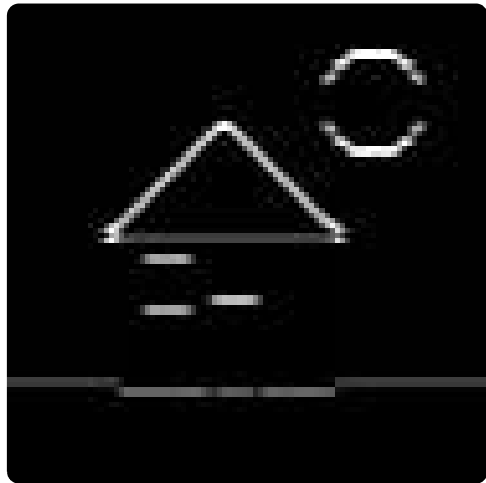
Sobel x subtracts the left from the right. Uniform patches cancel to zero; a brighter right side gives a positive response. Reverse the contrast, and the sign reverses.



Input image



Sobel x · vertical edges



Sobel y · horizontal edges

Edge previews show response magnitude, scaled for display. The calculation below retains the sign.

3.1 One patch becomes one number

- 1
- Multiply matching positions. Each patch value meets one filter weight. This is elementwise multiplication, not matrix multiplication.
- 2
- Add the products and one bias. Nine products from a 3 × 3 patch become one output value.
- 3
- Move and repeat. Write the result at the matching output position. Reuse the same weights and bias.

Input · 6 × 6 · values 0 or 1

0	0	0	0	0	0
0	0	1	1	0	0
0	0	1	1	0	0
0	0	1	1	0	0
0	0	1	1	0	0
0	0	0	0	0	0

Filter · 3 × 3 · fixed Sobel weights

-1	0	1
-2	0	2
-1	0	1

Patch × filter · multiply each pair

0 × -1 = 0	0 × 0 = 0	0 × 1 = 0
0 × -2 = 0	0 × 0 = 0	1 × 2 = 2
0 × -1 = 0	0 × 0 = 0	1 × 1 = 1

Sum + 0 = 3

Feature map · 4 × 4

3	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.

■ Positive ■ Negative

Inspect this output: begin with a running sum of 0.

Amber links one input, its weight and their product. Finish all nine products, then add the bias.

Classical image processing

People designed filters such as Sobel. The filter values were fixed.

Convolutional neural networks

Weights and biases are trainable.

Backpropagation combines gradients from every use of a shared weight. The task determines useful detectors; they need not resemble Sobel.

Convolution provides feature maps with shared detectors. One grayscale 3×3 filter needs only 9 weights + 1 bias = 10 parameters, regardless of the number of positions.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · A feature map records...

- ☐ Response by location
- ☐ Class labels
- ☐ Filter weights

► [Show reasoning](#)

Numerical · Patch [1, 2], weights [2, −1], bias 1: output?

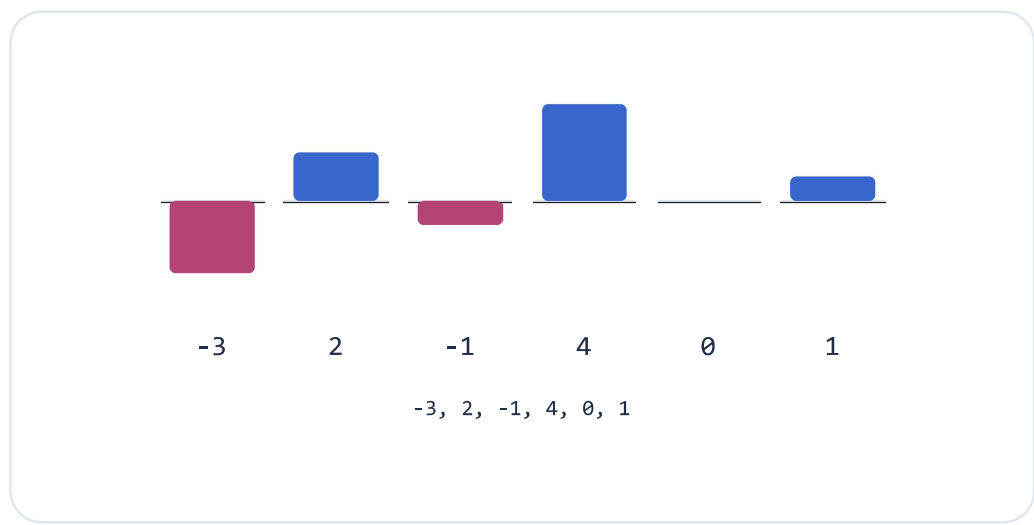
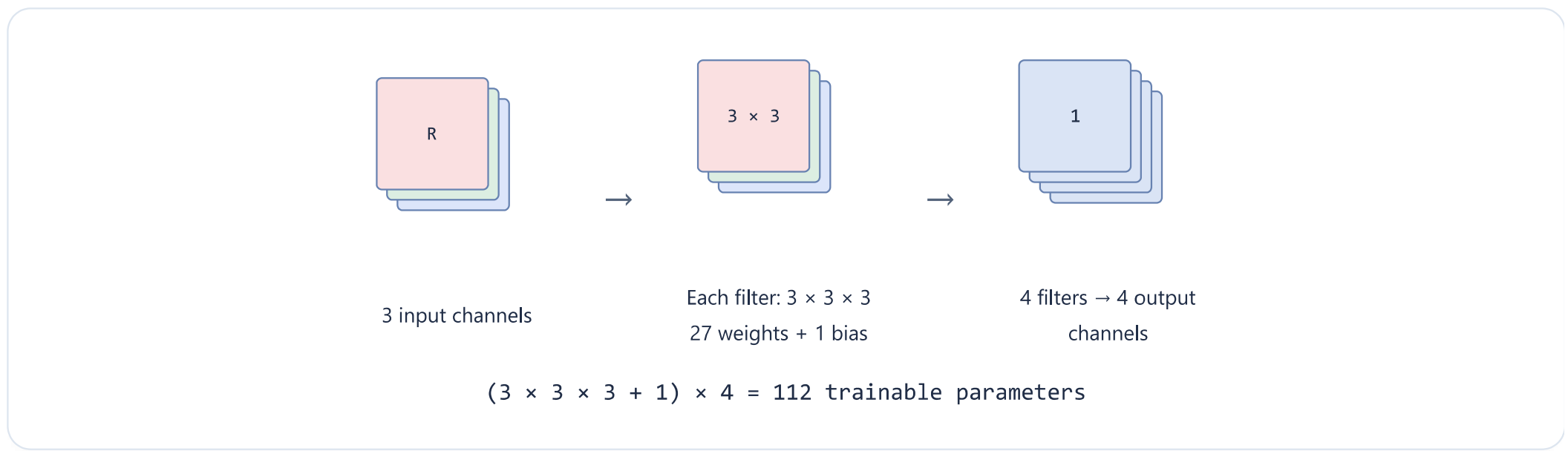
Answer

04 / BUILDING FEATURES

Across channels. Through a nonlinearity.

Why not stop after finding edges?

Each filter spans all input channels. Sum those responses to form one output feature map.



Keep positives. Zero negatives.

ReLU keeps positive responses and zeros negatives. It adds nonlinearity; stacking only affine layers still gives an affine transformation.

ReLU preserves height, width and channels.

Later layers see more.

The receptive field grows with depth: later features combine evidence from larger input regions.

Two 3×3 convolutions, stride 1
 $\rightarrow 5 \times 5$ receptive field

Nonlinearity lets stacked layers model complex relationships; ReLU changes values, not shape.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · ReLU adds...

☐ More channels

☐ Trainable weights

☐ Nonlinearity

► [Show reasoning](#)

Numerical · Sum after ReLU(−2, 0, 3)?

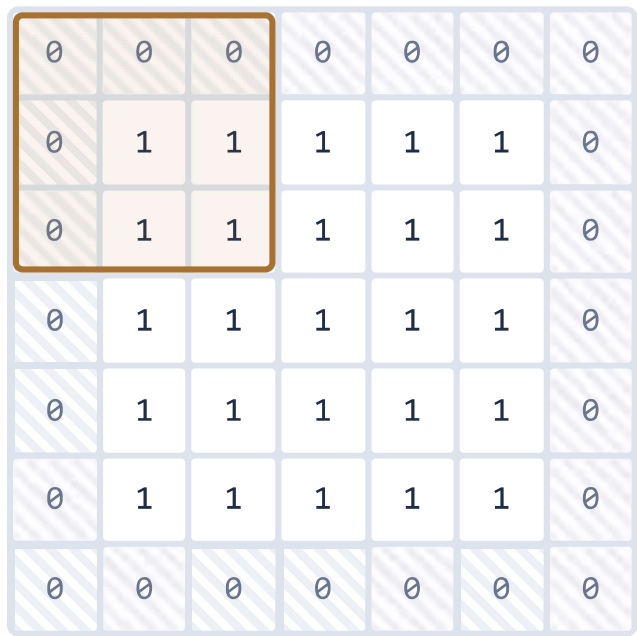
Answer

05 / CONTROLLING SHAPE

Padding, stride and output size.

How many places can the filter fit?

Padding adds a border. **Stride** is the distance the window moves. Neither introduces trainable weights.



5 × 5 input · 3 × 3 filter
Striped cells are zero padding.

VALID WINDOW POSITIONS

5 × 5 spatial output

$$\text{floor}((5 + 2 \times 1 - 3) / 1) + 1 = 5$$

Padding 1 and stride 1 preserve size for a 3 × 3 filter. Larger stride usually gives a smaller output.

Subtract the kernel to find available travel. Divide by stride to count moves. Round down; add 1 for the starting position.

$$\text{Output size} = \text{floor}((\text{input} + 2 \times \text{padding} - \text{kernel}) / \text{stride}) + 1$$

Apply separately to height and width. Padding is per side. Floor means round down.
Standard windows: no dilation; equal padding on opposite sides.

SOLVED / CONVOLUTION

32 × 32 × 3 → 8 filters

Kernel 3 · stride 2 · padding 1

$$\text{floor}((32 + 2 - 3) / 2) + 1 = 16$$

16 × 16 × 8

Padding and stride control spatial size. The number of filters controls output channels.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · Larger stride usually gives...

- ☐ Smaller spatial output
- ☐ More channels
- ☐ More weights

► Show reasoning

Numerical · 28 × 28; kernel 5, stride 2, padding 1: side?

Answer

× same × 6

Pooling reduces spatial size.

Do we need every exact feature location?

Pooling applies a fixed summary rule independently to each channel. Standard max and average pooling have no trainable weights or biases.

<u>Max pooling</u>	<u>Average pooling</u>	<u>Global average pooling</u>
Keeps the strongest response. Preserves prominent features; discards weaker ones.	Keeps local mean strength. Smooths responses; isolated peaks become weaker.	Averages each entire feature map into one value. Removes spatial layout; can replace flattening.

Input · 4 × 4 · one channel

1	3	2	4
5	6	1	8
9	2	0	1
3	4	5	2

Output · 2 × 2 · same channel

6	.
.	.

$\max(1, 3, 5, 6) = 6$

2 × 2 window · stride 2 · padding 0

Pooling provides compact summaries while preserving channels. The same output-size formula applies; use the pooling window as the kernel.

Pooling trades spatial detail for a compact representation. It does not guarantee translation invariance.

SOLVED / POOLING

28 × 28 × 6

Window 2 · stride 2 · padding 0

$\text{floor}((28 - 2) / 2) + 1 = 14$

14 × 14 × 6

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · Which keeps a local peak?

☐ Max pooling

☐ Average pooling

☐ Global average

Numerical · 15 × 15; window 3, stride 2, padding 1: side?

Answer

× same × 8

► Show reasoning

Fully connected: combine the evidence.

How do detected features become a class?

Flatten rearranges feature values into a vector; it learns nothing. In a **fully connected (dense) layer**, every neuron uses every input, with its own weights and bias.

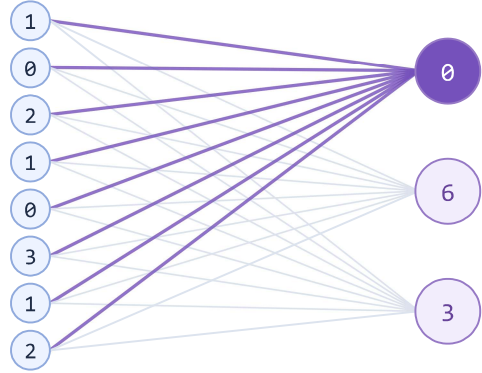
Two 2 × 2 feature maps → 8 values

1021

0312

Two feature maps · the same 8 values

Neuron 1: $1 \times 1 + 0 \times 0 + 2 \times 0.5 + 1 \times 0 + 0 \times 0 + 3 \times (-1) + 1 \times 1 + 2 \times 0 + \text{bias } 0 = 0$



Each neuron computes **weighted inputs + bias**. Hidden dense layers usually add an activation; a final classifier produces logits. Softmax converts these scores into class probabilities.

Parameters = (inputs + 1) × neurons

Fully connected layers combine extracted evidence into predictions; their weights are learned.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · Flatten learns new weights?

☐ No

☐ One per value

☐ One per channel

► [Show reasoning](#)

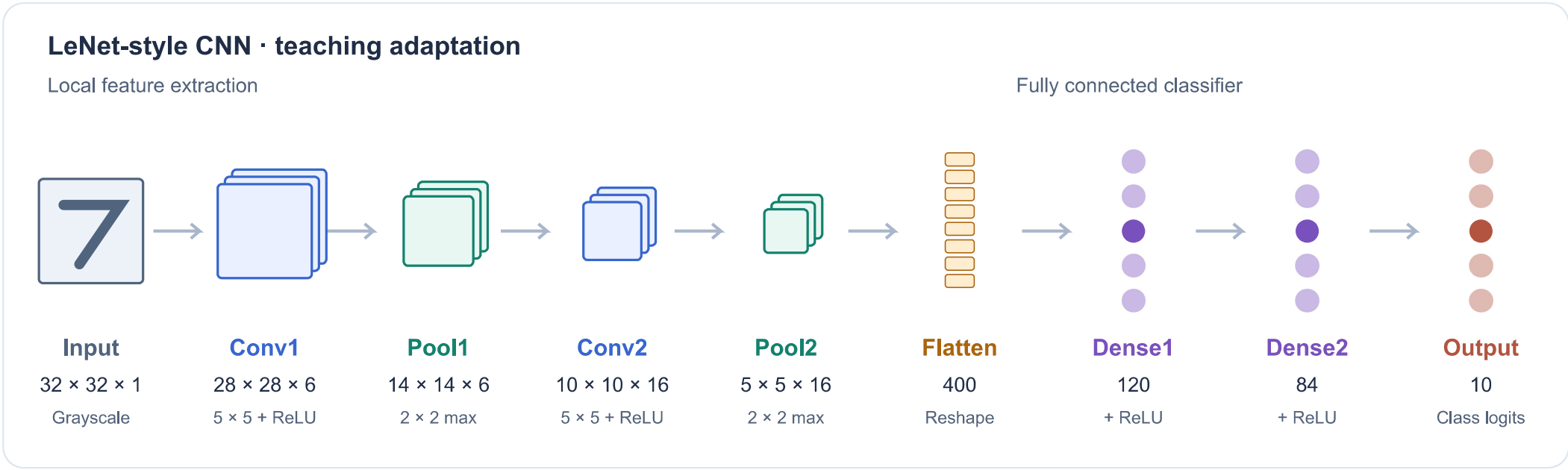
Numerical · 8 inputs → 3 dense neurons: parameters?

Answer

LeNet: a landmark in recognising digits.

Can these simple operations recognise handwriting?

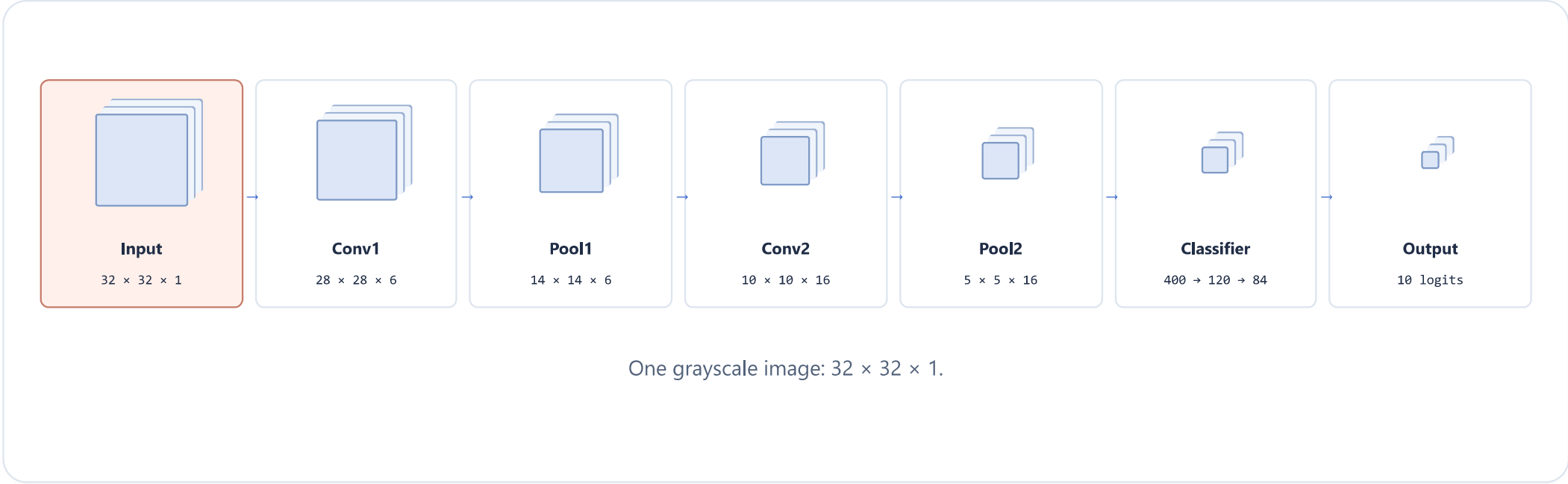
LeNet-5, described by LeCun and colleagues in 1998, learned handwritten digit recognition. Related convolutional systems read bank cheques—an early practical success for learned visual features. [Original paper](#).



Read left to right: extract local features, reduce spatial size, then combine features into class scores.

We trace a **teaching adaptation**: ReLU, max pooling, full channel connectivity and a dense classifier differ from historical LeNet-5. Conv: kernel 5, stride 1, padding 0. Pool: window 2, stride 2. Biases included.

Now trace the shapes, then count the weights.



Convolution parameters			Dense parameters
(kernel height × kernel width × input channels + 1) × filters			(inputs + 1) × outputs
" + 1 " includes one bias per filter or neuron. ReLU, standard pooling and flatten add zero parameters.			
Layer	Output shape	Parameter calculation	Parameters
Conv1 + ReLU	28 × 28 × 6	(5 × 5 × 1 + 1) × 6	156
Pool1	14 × 14 × 6	No learned weights	0
Conv2 + ReLU	10 × 10 × 16	(5 × 5 × 6 + 1) × 16	Your turn
Pool2	5 × 5 × 16	No learned weights	0
Total trainable parameters			Total

Layer	Output shape	Parameter calculation	Parameters
Flatten	Your turn	$5 \times 5 \times 16$ values; reshape only	0
Dense1 + ReLU	120	$(\text{flattened length} + 1) \times 120$	Your turn
Dense2 + ReLU	84	$(120 + 1) \times 84$	10,164
Output logits	10	$(84 + 1) \times 10$	850
Total trainable parameters			

Activations ≠ parameters. Conv1 produces 4,704 values per image using just 156 shared parameters. Larger input images increase computation, not the weights of a fixed convolution.

► Why do the dense layers contain most of the weights?

LeNet combines the operations into a trainable image classifier.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · Convolution before dense learns...

- ☐ Spatial features
- ☐ Class names directly
- ☐ Only image size

Numerical · Flatten $5 \times 5 \times 16$: vector length?

Answer

► Show reasoning

09 / TAKE IT WITH YOU

Five operations. One classifier.

Which operation does each job?

Convolution

Learn local, shared filters.

ReLU

Add nonlinearity.

Pooling

Summarize neighborhoods.

Flatten

Rearrange feature values.

Dense

Combine features into scores.

QUICK CHECK · UNDERSTANDING + CALCULATION

MCQ · Which has trainable weights?

- ☐ ReLU
- ☐ Max pooling
- ☐ Dense

Numerical · Parameters in ReLU + standard max pooling?

Answer

► Show reasoning

